ME-2 Digital Twin Baseline Construction

Week 5: Static Baseline Study

Blayne Montaque

Spring 2026

1 Purpose

The purpose of this stage is to develop and evaluate the first static steady-state baseline model for the behavioral digital twin. Using experimentally measured or representative baseline data, the MATLAB implementation establishes the expected relationship between applied load and measured structural response under static conditions.

This stage builds on the previous weeks involving digital twin scope definition, sensor calibration planning, data familiarization, and expected-response relationship development. The resulting model provides the first quantitative baseline used to define normal structural behavior within the digital twin framework.

The accompanying MATLAB file, `static_ss_model.m`, performs the model fitting, residual analysis, statistical evaluation, and visualization required for the static baseline study.

2 Background

The behavioral digital twin being developed in this project is intended to represent expected structural behavior using experimentally measured sensor data rather than a full physics-based finite element model.

Earlier stages of the project established:

- The scope and purpose of the digital twin
- Sensor calibration procedures
- Data quality review methods
- Expected-response relationships between inputs and outputs

The next step is to construct the first usable baseline model from the measured data. The static steady-state model focuses specifically on the relationship between applied load and measured load response under non-vibrating conditions.

This model serves as the foundation for later dynamic and temperature-conditioned models.

3 Modeling Objective

The static baseline model is intended to answer the following question:

Given a known applied load, what load response should the structure produce under normal conditions?

For the current operating range, the aluminum structural member is assumed to behave approximately linearly within the elastic region. Under this assumption, the measured load response should scale proportionally with the applied load.

The static model is written as:

$$\hat{y} = ax + b$$

where:

- x is the applied load,
- $\hat{y}$ is the predicted measured response,
- a is the fitted slope,
- b is the fitted intercept.

The slope describes the sensitivity of the measured response to loading, while the intercept accounts for baseline offset and sensor bias.

4 MATLAB Implementation Overview

The static baseline model is implemented in the MATLAB script:

`static_ss_model.m`

The script performs the following tasks:

1. Imports or generates static load-response data

2. Fits a linear steady-state model
3. Computes residuals and model statistics
4. Defines statistical tolerance bounds
5. Generates plots for visualization and evaluation

The current implementation uses a representative dummy dataset to allow the workflow to be tested before the final experimental dataset is connected.

In the completed workflow, the script can directly import experimental measurements from an Excel file such as:

`static_baseline_data.xlsx`

using measured columns containing:

- Applied load
- Measured load response

5 Static Data Structure

The model uses two primary variables.

Table 1: Static Model Variables

Variable	**MATLAB Variable**	**Description**
Applied Load	`x`	Controlled load applied to the structure
Measured Load	`y`	Calibrated load cell response

The applied load represents the controlled system input, while the measured load represents the observed structural response.

6 Linear Model Fitting

The MATLAB implementation uses a first-order polynomial fit to estimate the static relationship between applied load and measured response.

The fitting process is implemented using:

$$p = \texttt{polyfit}(x, y, 1)$$

This returns the slope and intercept coefficients of the fitted line.

The predicted response is then evaluated using:

$$\hat{y} = \texttt{polyval}(p, x)$$

A first-order model is appropriate at this stage because the structure is expected to remain within the linear elastic operating range during testing.

7 Residual Analysis

After fitting the model, the residual error is calculated at each point:

$$r_i = y_i - \hat{y}_i$$

where:

- r_i is the residual,
- y_i is the measured load response,
- $\hat{y}_i$ is the predicted load response.

The residual represents the difference between the physical measurement and the expected baseline response predicted by the model.

Residuals are important because they form the basis for future anomaly detection within the digital twin framework. Ideally, the residuals should remain small and randomly distributed around zero.

Large residuals may indicate:

- Sensor noise
- Calibration error
- Drift
- Structural change
- Unmodeled nonlinear behavior

8 Model Evaluation Metrics

Several statistical quantities are computed to evaluate the quality of the static model.

Table 2: Model Evaluation Metrics

Metric	**MATLAB Variable**	**Purpose**
Coefficient of Determination	`R2`	Measures goodness of fit
Root Mean Square Error	`RMSE`	Measures average model error magnitude
Mean Residual	`muRes`	Indicates residual bias
Residual Standard Deviation	`sigRes`	Measures residual spread
Tolerance Band	`tol`	Defines acceptable response deviation

The coefficient of determination is computed as:

$$R^2 = 1 - \frac{SS_{\text{res}}}{SS_{\text{tot}}}$$

where:

$$SS_{\text{res}} = \sum (y_i - \hat{y}_i)^2$$

and:

$$SS_{\text{tot}} = \sum (y_i - \bar{y})^2$$

The root mean square error is calculated using:

$$RMSE = \sqrt{\frac{1}{N} \sum_{i=1}^{N} r_i^2}$$

A high R^2 value and low RMSE indicate that the model closely matches the measured baseline data.

9 Tolerance Envelope

The model includes a statistical tolerance envelope around the fitted response.

The tolerance band is defined as:

$$\text{tol} = k\sigma_r$$

where:

- k is a selected multiplier,
- σ_r is the standard deviation of the residuals.

The current implementation uses:

$$k = 2$$

The upper and lower limits are therefore:

$$\hat{y}_{upper} = \hat{y} + \text{tol}$$

$$\hat{y}_{lower} = \hat{y} - \text{tol}$$

These limits define the acceptable range of normal structural response for the static baseline condition.

10 Generated Visualizations

The MATLAB script generates two primary plots for evaluating the model.

Table 3: Generated Model Plots

Plot	**Purpose**
Static Model Fit	Compares measured data against the fitted model and tolerance envelope
Residual Plot	Evaluates residual behavior relative to zero and tolerance limits

The static fit plot allows visual comparison between measured data and expected response.

The residual plot helps determine whether the model errors remain random or show systematic trends with increasing load.

A well-behaved static model should produce residuals that:

- Remain centered near zero

- Show no strong trend with load
- Stay mostly within tolerance limits

11 Connection to the Digital Twin

The static baseline study represents the first implemented predictive component of the behavioral digital twin.

The resulting model contributes to:

- Expected load-response prediction
- Static response envelopes
- Residual generation
- Baseline behavioral characterization
- Future anomaly detection workflows

The general residual relationship used by the twin is:

$$r(t) = y_{\text{measured}}(t) - y_{\text{predicted}}(t)$$

At this stage, the prediction depends only on applied load. Later stages of the project can extend the model to include:

- Dynamic vibration response
- Temperature compensation
- Frequency-domain features
- Multi-input behavioral relationships

12 Limitations

The current implementation intentionally uses a simplified model. Several limitations should be acknowledged:

- The current model assumes linear structural behavior.
- Dynamic vibration effects are not yet included.

- Temperature effects are not explicitly modeled.
- The current implementation uses representative baseline data rather than the final experimental dataset.
- Only steady-state static behavior is considered.

These limitations are acceptable for the current stage because the goal is to establish an interpretable baseline before introducing more complex coupled behavior.

13 Summary

Week 5 introduces the first implemented static baseline model for the behavioral digital twin. The MATLAB script `static_ss_model.m` fits a linear relationship between applied load and measured load response, computes residuals and statistical metrics, and generates tolerance bounds representing expected static behavior.

This study establishes the first operational predictive model within the digital twin framework and provides the foundation for future dynamic, temperature-conditioned, and anomaly-detection modeling stages.

A Appendix A: MATLAB Implementation

The complete MATLAB implementation used for the static baseline study is provided below. The script performs the following operations:

- Generates or imports static baseline data
- Fits a linear steady-state model
- Computes residuals and statistical metrics
- Defines tolerance bounds
- Produces plots for model evaluation

The implementation is contained in the file:

`static_ss_model.m`

```
%
    ------------------------------------------------------------------------

% Static Model Fit from Experimental Data
%
% This script:
% 1. Reads applied load and measured load data from an Excel file
% 2. Fits a linear model: y = a*x + b
% 3. Computes basic statistics (R^2, RMSE, residuals, tolerance)
% 4. Plots:
%    - Measured data with fitted model and tolerance band
%    - Residuals vs load
%
% Required Excel columns:
%   AppliedLoad
%   MeasuredLoad
%
    ------------------------------------------------------------------------


clc;
clear;
close all;

```

```
%% USER SETTINGS

% fileName = 'static_baseline_data.xlsx';
% sheetName = 1;

kTol = 2;

%% DUMMY DATASET
% Replace with experimental Excel import later

rng(1);

x = (0:2:20)';

trueSlope     = 0.98;
trueIntercept = 0.12;
noiseLevel    = 0.35;

y = trueSlope*x + trueIntercept ...
    + noiseLevel*randn(size(x));

%% OPTIONAL EXCEL IMPORT

% tbl = readtable(fileName, 'Sheet', sheetName);
%
% x = tbl.AppliedLoad;
% y = tbl.MeasuredLoad;
%
% validIdx = ~(isnan(x) | isnan(y));
%
% x = x(validIdx);
% y = y(validIdx);

%% ENSURE COLUMN VECTORS

x = x(:);
y = y(:);

%% LINEAR MODEL FIT

p = polyfit(x, y, 1);
```

```
62
63 a = p(1);
64 b = p(2);
65
66 yFit = polyval(p, x);
67
68 %% RESIDUALS
69
70 res = y - yFit;
71
72 %% STATISTICS
73
74 SSres = sum(res.^2);
75
76 SStot = sum((y - mean(y)).^2);
77
78 R2 = 1 - SSres/SStot;
79
80 RMSE = sqrt(mean(res.^2));
81
82 muRes = mean(res);
83
84 sigRes = std(res);
85
86 tol = kTol * sigRes;
87
88 %% SORT FOR PLOTTING
89
90 [xSort, idx] = sort(x);
91
92 yFitSort = yFit(idx);
93
94 yUpperSort = yFitSort + tol;
95
96 yLowerSort = yFitSort - tol;
97
98 %% DISPLAY RESULTS
99
100 fprintf('\n');
101
102 fprintf('----------------------------------------\n');
```

```
fprintf('STATIC MODEL RESULTS\n');
fprintf('----------------------------------------\n\n');

fprintf('Model Form:\n');
fprintf('    y = a*x + b\n\n');

fprintf('Coefficients:\n');
fprintf('    a = %.6f\n', a);
fprintf('    b = %.6f\n\n', b);

fprintf('Statistics:\n');
fprintf('    R^2           = %.6f\n', R2);
fprintf('    RMSE          = %.6f\n', RMSE);
fprintf('    Mean Residual = %.6f\n', muRes);
fprintf('    Std Residual  = %.6f\n', sigRes);
fprintf('    Tolerance     = +/- %.6f\n\n', tol);

%% FIGURE

figure('Name','Static Model Results', ...
       'NumberTitle','off', ...
       'Color','w', ...
       'Units','normalized', ...
       'Position',[0.1 0.2 0.8 0.5]);

%% STATIC MODEL FIT

subplot(1,2,1)

scatter(x, y, 55, 'filled');

hold on;

plot(xSort, yFitSort, 'LineWidth', 2);

plot(xSort, yUpperSort, '--', 'LineWidth', 1.5);

plot(xSort, yLowerSort, '--', 'LineWidth', 1.5);

xlabel('Applied Load');

```

```
144 ylabel('Measured Load');
145
146 title('Static Model Fit');
147
148 legend('Measured Data', ...
149        'Linear Fit', ...
150        'Upper Tolerance', ...
151        'Lower Tolerance', ...
152        'Location', 'best');
153
154 grid on;
155
156 box on;
157
158 %% RESIDUAL PLOT
159
160 subplot(1,2,2)
161
162 scatter(x, res, 55, 'filled');
163
164 hold on;
165
166 yline(0, 'LineWidth', 1.5);
167
168 yline(tol, '--', 'LineWidth', 1.5);
169
170 yline(-tol, '--', 'LineWidth', 1.5);
171
172 xlabel('Applied Load');
173
174 ylabel('Residual');
175
176 title('Residual Plot');
177
178 legend('Residuals', ...
179        'Zero Line', ...
180        'Upper Tolerance', ...
181        'Lower Tolerance', ...
182        'Location', 'best');
183
184 grid on;
```

```
185
186 box on;
```